

Program No. 6

Client–Server Communication Using TCP

Aim

To implement TCP-based client-server communication in which the client inputs an integer N , generates a square matrix of order N populated with random numbers in the range $[1, 50]$, and sends it to the server. The server examines the matrix and identifies its type — Upper Triangular, Lower Triangular, Diagonal, or Normal — and returns this type as a string to the client, which then displays it.

Algorithm – Client

1. Start the client program and import the required modules: `socket` (for network communication), `pickle` (for serializing the matrix), and `random` (for generating matrix values).
2. Create a TCP socket using `socket.AF_INET` (IPv4 addressing) and `socket.SOCK_STREAM` (connection-oriented TCP communication).
3. Connect the socket to the server by specifying the server's IP address and port number, establishing a TCP connection.
4. Prompt the user to enter the order N , which determines the size of the square matrix.
5. Generate an $N \times N$ matrix with random integer values in the range $[1, 50]$.
6. Randomly select a target matrix category (upper triangular, lower triangular, diagonal, or normal) and set the corresponding cells of the matrix to zero so that the chosen structure is actually formed.
7. Display the generated matrix on the client's console so the user can see what is being sent.
8. Serialize the matrix together with its order N into a byte stream using `pickle.dumps()`, so it can be transmitted over the network.
9. Send the serialized data to the server through the connected socket.

10. Wait to receive the server's reply, which contains the identified matrix type encoded as a string, and decode it.
11. Display the received matrix type to the user, then close the client socket to terminate the connection.

Algorithm – Server

1. Start the server program and import the required modules: socket and pickle.
2. Create a TCP socket using socket.AF_INET and socket.SOCK_STREAM.
3. Bind the socket to address 0.0.0.0 (all available network interfaces) and a fixed port number, so the server can be reached by the client.
4. Put the socket into listening mode with a backlog of 5, allowing it to queue incoming connection requests.
5. Print a message indicating that the server is waiting for a client to connect.
6. Accept an incoming connection request, which returns a new socket object for that client along with the client's address; print a confirmation message showing that a client has connected, along with its address, so the server's progress is visible.
7. Receive the byte stream sent by the client and deserialize it using pickle.loads() to recover the order N and the matrix; print the received order and the matrix itself on the server console to confirm that the data arrived correctly.
8. Initialize three boolean flags — upper, lower, and diagonal — all set to True, to track the possible classification of the matrix.
9. Traverse every element of the matrix using nested loops. If an element below the main diagonal (row index > column index) is non-zero, mark upper as False. If an element above the main diagonal (row index < column index) is non-zero, mark lower as False. If any off-diagonal element is non-zero, mark diagonal as False.
10. After scanning the full matrix, decide the matrix type from the flags: Diagonal Matrix if diagonal is still True, otherwise Upper Triangular Matrix if upper is True, otherwise Lower Triangular Matrix if lower is True, otherwise Normal Matrix.

11. Encode the resulting matrix-type string, print it on the server console as confirmation of what is being sent, and then send it back to the client over the same connection.
12. Close the connection socket and then close the server socket to release the resources.

Sample Result

Client Side

Enter order of matrix: 3

Matrix:

5 12 8

0 3 17

0 0 9

Matrix Type: Upper Triangular Matrix

Server Side

Server waiting for connection...

Connected by client: ('127.0.0.1',

54821) Received matrix of order 3 from

client: 5 12 8

0 3 17

0 0 9

Identified matrix type: Upper Triangular Matrix

Sending result to client...